

Язык программирования С

Лекция 1: языки высокого уровня, программы из нескольких файлов

Лисица Никита

2026

Напоминание из курса информатики :)

- Всё, с чем умеет работать компьютер – это числа:

Напоминание из курса информатики :)

- Всё, с чем умеет работать компьютер – это числа:
 - Текст – последовательность чисел, интерпретируемая посредством кодировки (напр. в ASCII символ 'A' это число 65)

Напоминание из курса информатики :)

- Всё, с чем умеет работать компьютер – это числа:
 - Текст – последовательность чисел, интерпретируемая посредством кодировки (напр. в ASCII символ 'A' это число 65)
 - Картинка – последовательность чисел, задающих цвета (RGB – красный, зелёный, синий)

Напоминание из курса информатики :)

- Всё, с чем умеет работать компьютер – это числа:
 - Текст – последовательность чисел, интерпретируемая посредством кодировки (напр. в ASCII символ 'A' это число 65)
 - Картинка – последовательность чисел, задающих цвета (RGB – красный, зелёный, синий)
 - Звук – последовательность чисел, задающих звуковую волну (часто 16-битные, от -32768 до 32767)

Напоминание из курса информатики :)

- Всё, с чем умеет работать компьютер – это числа:
 - Текст – последовательность чисел, интерпретируемая посредством кодировки (напр. в ASCII символ 'A' это число 65)
 - Картинка – последовательность чисел, задающих цвета (RGB – красный, зелёный, синий)
 - Звук – последовательность чисел, задающих звуковую волну (часто 16-битные, от -32768 до 32767)
 - Программа – последовательность чисел, задающих команды для исполнения процессору

Напоминание из курса информатики :)

- Всё, с чем умеет работать компьютер – это числа:
 - Текст – последовательность чисел, интерпретируемая посредством кодировки (напр. в ASCII символ 'A' это число 65)
 - Картинка – последовательность чисел, задающих цвета (RGB – красный, зелёный, синий)
 - Звук – последовательность чисел, задающих звуковую волну (часто 16-битные, от -32768 до 32767)
 - Программа – последовательность чисел, задающих команды для исполнения процессору
- Числа хранятся в двоичной системе счисления (есть ток / нет тока)

Напоминание из курса информатики :)

- Всё, что умеет делать процессор – операции над числами (арифметические, логические) и работать с периферийными устройствами (память, жёсткий диск, видеокарта, экран)

Напоминание из курса информатики :)

- Всё, что умеет делать процессор – операции над числами (арифметические, логические) и работать с периферийными устройствами (память, жёсткий диск, видеокарта, экран)
 - Арифметическая операция: прочитать аргументы из памяти в процессор, выполнить операцию, записать результат в память

Напоминание из курса информатики :)

- Всё, что умеет делать процессор – операции над числами (арифметические, логические) и работать с периферийными устройствами (память, жёсткий диск, видеокарта, экран)
 - Арифметическая операция: прочитать аргументы из памяти в процессор, выполнить операцию, записать результат в память
 - Вывести картинку: отправить цвета пикселей на экран

Напоминание из курса информатики :)

- Всё, что умеет делать процессор – операции над числами (арифметические, логические) и работать с периферийными устройствами (память, жёсткий диск, видеокарта, экран)
 - Арифметическая операция: прочитать аргументы из памяти в процессор, выполнить операцию, записать результат в память
 - Вывести картинку: отправить цвета пикселей на экран
 - Записать файл: послать данные и их положение на диске в контроллер жесткого диска

- У процессора есть

- У процессора есть
 - Набор поддерживаемых команд

- У процессора есть
 - Набор поддерживаемых команд
 - Регистры – ячейки памяти внутри процессора (очень быстрые чтение/запись)

- У процессора есть
 - Набор поддерживаемых команд
 - Регистры – ячейки памяти внутри процессора (очень быстрые чтение/запись)
- Всё это вместо называется *архитектурой процессора*: x86, x86-64 (AMD64), Aarch64 (ARM64), ...

- Как могла бы выглядеть (условно) программа (для гипотетической архитектуры процессора):

Программа в машинных кодах

- Как могла бы выглядеть (условно) программа (для гипотетической архитектуры процессора):

```
45 100 1    // Загрузить число из ячейки 100 в регистр R1
45 300 2    // Загрузить число из ячейки 300 в регистр R2
27 1 2 3    // Сложить R1 и R2 и записать результат в R3
46 500 3    // Записать число из R3 в ячейку памяти 500
```

Программа в машинных кодах

- Как могла бы выглядеть (условно) программа (для гипотетической архитектуры процессора):

```
45 100 1    // Загрузить число из ячейки 100 в регистр R1
45 300 2    // Загрузить число из ячейки 300 в регистр R2
27 1 2 3    // Сложить R1 и R2 и записать результат в R3
46 500 3    // Записать число из R3 в ячейку памяти 500
```

- 27 – код команды ‘сложить два числа’
- 45 – код команды ‘загрузить из памяти’
- 46 – код команды ‘записать в память’

- Программировать в таком виде очень неудобно:

- Программировать в таком виде очень неудобно:
 - Нужно помнить номера команд и порядок их аргументов

- Программировать в таком виде очень неудобно:
 - Нужно помнить номера команд и порядок их аргументов
 - Нужно помнить, какие регистры заняты (и чем)

- Программировать в таком виде очень неудобно:
 - Нужно помнить номера команд и порядок их аргументов
 - Нужно помнить, какие регистры заняты (и чем)
 - Нужно помнить, какие ячейки памяти заняты (и чем)

- Программировать в таком виде очень неудобно:
 - Нужно помнить номера команд и порядок их аргументов
 - Нужно помнить, какие регистры заняты (и чем)
 - Нужно помнить, какие ячейки памяти заняты (и чем)
 - Программа под одну архитектуру не будет работать на другой

- Ассемблер (от англ. assembler – сборщик) – программа, транслирующая текст на особом языке (языке ассемблера) в машинный код

Язык ассемблера

- Ассемблер (от англ. assembler – сборщик) – программа, транслирующая текст на особом языке (языке ассемблера) в машинный код
- У команд и регистров есть понятные, запоминающиеся имена

Язык ассемблера

- Ассемблер (от англ. assembler – сборщик) – программа, транслирующая текст на особом языке (языке ассемблера) в машинный код
- У команд и регистров есть понятные, запоминающиеся имена
- Ячейкам памяти можно давать свои имена

Язык ассемблера

- Ассемблер (от англ. assembler – сборщик) – программа, транслирующая текст на особом языке (языке ассемблера) в машинный код
- У команд и регистров есть понятные, запоминающиеся имена
- Ячейкам памяти можно давать свои имена

```
label 100 x
label 300 y
label 500 z

load x reg1
load y reg2
add reg1 reg2 reg3
store z reg3
```

- На языке ассемблера программировать гораздо удобнее, чем в машинных кодах, но некоторые проблемы остаются:

- На языке ассемблера программировать гораздо удобнее, чем в машинных кодах, но некоторые проблемы остаются:
 - Нет удобных управляющих конструкций (только `jump`'ы – 'прыжок' с одной инструкции на другую)

- На языке ассемблера программировать гораздо удобнее, чем в машинных кодах, но некоторые проблемы остаются:
 - Нет удобных управляющих конструкций (только `jump`'ы – 'прыжок' с одной инструкции на другую)
 - Тяжело писать сложные программы

- На языке ассемблера программировать гораздо удобнее, чем в машинных кодах, но некоторые проблемы остаются:
 - Нет удобных управляющих конструкций (только jump'ы – 'прыжок' с одной инструкции на другую)
 - Тяжело писать сложные программы
 - Всё ещё нужно помнить, какие регистры чем заняты

- На языке ассемблера программировать гораздо удобнее, чем в машинных кодах, но некоторые проблемы остаются:
 - Нет удобных управляющих конструкций (только jump'ы – 'прыжок' с одной инструкции на другую)
 - Тяжело писать сложные программы
 - Всё ещё нужно помнить, какие регистры чем заняты
 - Программа всё ещё заточена под конкретную архитектуру

```
int x = 10;  
int y = 20;  
int z = x + y;
```

```
int x = 10;  
int y = 20;  
int z = x + y;
```

- Компилятор – программа, преобразующая текст на языке программирования в язык ассемблера (и затем в машинный код)

```
int x = 10;  
int y = 20;  
int z = x + y;
```

- Компилятор – программа, преобразующая текст на языке программирования в язык ассемблера (и затем в машинный код)
 - Сам подбирает нужные команды, ячейки памяти, номера регистров, и т.д.

```
int x = 10;  
int y = 20;  
int z = x + y;
```

- Компилятор – программа, преобразующая текст на языке программирования в язык ассемблера (и затем в машинный код)
 - Сам подбирает нужные команды, ячейки памяти, номера регистров, и т.д.
 - Сам разбирается с особенностями конкретной архитектуры процессора

```
int x = 10;  
int y = 20;  
int z = x + y;
```

- Компилятор – программа, преобразующая текст на языке программирования в язык ассемблера (и затем в машинный код)
 - Сам подбирает нужные команды, ячейки памяти, номера регистров, и т.д.
 - Сам разбирается с особенностями конкретной архитектуры процессора
 - Оптимизирует подбор команд, чтобы программа работала быстрее

- Появился в 1972 в AT&T Bell Labs с целью переписать ядро операционной системы Unix с языка ассемблера на кросс-платформенный язык

- Появился в 1972 в AT&T Bell Labs с целью переписать ядро операционной системы Unix с языка ассемблера на кросс-платформенный язык
- Был вдохновлён языками ALGOL 60, BCPL, B

- Появился в 1972 в AT&T Bell Labs с целью переписать ядро операционной системы Unix с языка ассемблера на кросс-платформенный язык
- Был вдохновлён языками ALGOL 60, BCPL, B
- Обрёл популярность после переписывания ядра Unix на C, и стал де-факто стандартом для низкоуровневых библиотек и интерфейсов

- Появился в 1972 в AT&T Bell Labs с целью переписать ядро операционной системы Unix с языка ассемблера на кросс-платформенный язык
- Был вдохновлён языками ALGOL 60, BCPL, B
- Обрёл популярность после переписывания ядра Unix на C, и стал де-факто стандартом для низкоуровневых библиотек и интерфейсов
- В современности, как правило, для поддержки новой архитектуры достаточно реализовать под неё компилятор C (или бэкенд LLVM)

- Зачем программу разбивают на части?

- Зачем программу разбивают на части?
 - Чем меньше код, тем его проще понять, тестировать, и отлаживать

- Зачем программу разбивают на части?
 - Чем меньше код, тем его проще понять, тестировать, и отлаживать
 - Над разными частями могут работать разные люди

- Зачем программу разбивают на части?
 - Чем меньше код, тем его проще понять, тестировать, и отлаживать
 - Над разными частями могут работать разные люди
 - Выделенные части кода легче переиспользовать в других проектах

Пример программы: игра в шахматы

- Файл с обработкой логики:
 - Функция 'проверить валидность хода'
 - Функция 'сделать ход'
 - Функция 'проверить выигрыш'

Пример программы: игра в шахматы

- Файл с обработкой логики:
 - Функция 'проверить валидность хода'
 - Функция 'сделать ход'
 - Функция 'проверить выигрыш'
- Файл с обработкой пользовательского ввода:
 - Функция 'узнать положение курсора мыши'
 - Функция 'проверить нажатость кнопки мыши'

Пример программы: игра в шахматы

- Файл с обработкой логики:
 - Функция 'проверить валидность хода'
 - Функция 'сделать ход'
 - Функция 'проверить выигрыш'
- Файл с обработкой пользовательского ввода:
 - Функция 'узнать положение курсора мыши'
 - Функция 'проверить нажатость кнопки мыши'
- Файл с выводом на экран:
 - Функция 'нарисовать шахматную доску'
 - Функция 'нарисовать шахматную фигуру'

Пример программы: сумма чисел

```
// sum.c
int sum(int x, int y) {
    return x + y;
}
```

```
// main.c
int main() {
    int a = 3, b = 5;
    int c = sum(a, b);
    return 0;
}
```

- Компилятор превращает исходный код в *объектный файл* – машинный код + пометки о том, где лежит код какой функции
 - `main.c` -> `main.o`
 - `sum.c` -> `sum.o`

Процесс сборки программы

- Компилятор превращает исходный код в *объектный файл* – машинный код + пометки о том, где лежит код какой функции
 - `main.c -> main.o`
 - `sum.c -> sum.o`
- *Линковщик* (компоновщик) превращает список объектных файлов в готовую программу
 - `main.o + sum.o -> sum.exe`

- В объектных файлах вызовы функций записаны по именам, так как на этом этапе неизвестны финальные адреса функций в программе

```
[10] load a reg1  
[11] load b reg2  
[12] call sum  
[13] store c reg3  
[14] return
```

- Линковщик собирает код всех функций вместе, назначает им адреса, и заменяет вызовы по имени на вызов по адресу

```
main:
[10] load a reg1
[11] load b reg2
[12] call [15]
[13] store c reg3
[14] return

sum:
[15] add reg1 reg2 reg3
[16] return
```

- Линковщик собирает код всех функций вместе, назначает им адреса, и заменяет вызовы по имени на вызов по адресу

```
main:
[10] load a reg1
[11] load b reg2
[12] call [15]
[13] store c reg3
[14] return

sum:
[15] add reg1 reg2 reg3
[16] return
```

- На уровне процессора имён функций *не существует*, есть только адреса инструкций

- Компиляция намного сложнее, чем линковка:

Процесс сборки программы

- Компиляция намного сложнее, чем линковка:
 - Компиляция: превратить управляющие конструкции в `jump`'ы, проверить типы, выделить регистры, при необходимости выгрузить в память промежуточные вычисления, подобрать эффективные инструкции, удалить лишние инструкции

Процесс сборки программы

- Компиляция намного сложнее, чем линковка:
 - Компиляция: превратить управляющие конструкции в `jump`'ы, проверить типы, выделить регистры, при необходимости выгрузить в память промежуточные вычисления, подобрать эффективные инструкции, удалить лишние инструкции
 - Линковка: склеить вместе готовый машинный код и заменить имена на адреса

Процесс сборки программы

- Компиляция намного сложнее, чем линковка:
 - Компиляция: превратить управляющие конструкции в `jump`'ы, проверить типы, выделить регистры, при необходимости выгрузить в память промежуточные вычисления, подобрать эффективные инструкции, удалить лишние инструкции
 - Линковка: склеить вместе готовый машинный код и заменить имена на адреса
- Как правило, программист в конкретный момент работает только с небольшим набором файлов (например, изменили функцию `sum`, не меняя `main`)

Процесс сборки программы

- Компиляция намного сложнее, чем линковка:
 - Компиляция: превратить управляющие конструкции в `jump`'ы, проверить типы, выделить регистры, при необходимости выгрузить в память промежуточные вычисления, подобрать эффективные инструкции, удалить лишние инструкции
 - Линковка: склеить вместе готовый машинный код и заменить имена на адреса
- Как правило, программист в конкретный момент работает только с небольшим набором файлов (например, изменили функцию `sum`, не меняя `main`)
- Перекомпилировать нужно только те файлы, в которых были изменения (например, только `sum.c`)

Процесс сборки программы

- Компиляция намного сложнее, чем линковка:
 - Компиляция: превратить управляющие конструкции в `jump`'ы, проверить типы, выделить регистры, при необходимости выгрузить в память промежуточные вычисления, подобрать эффективные инструкции, удалить лишние инструкции
 - Линковка: склеить вместе готовый машинный код и заменить имена на адреса
- Как правило, программист в конкретный момент работает только с небольшим набором файлов (например, изменили функцию `sum`, не меняя `main`)
- Перекомпилировать нужно только те файлы, в которых были изменения (например, только `sum.c`)
- Линковку всё равно придётся сделать целиком, но это дешёвая операция

- **Undefined reference:** вызов несуществующей функции (вызов есть, а функции нет)
- **Multiple definition:** несколько функций с одинаковым именем

- Линковщик и объектные файлы ничего не знают про типы и количество аргументов у функций

Линковщик и типы

- Линковщик и объектные файлы ничего не знают про типы и количество аргументов у функций

```
// sum.c
int sum(int x, int y) {
    return x + y;
}
```

```
// main.c
int main() {
    int a = 3, b = 5;
    int c = sum(a);
    return 0;
}
```

Линковщик и типы

- Линковщик и объектные файлы ничего не знают про типы и количество аргументов у функций

```
// sum.c
int sum(int x, int y) {
    return x + y;
}
```

```
// main.c
int main() {
    int a = 3, b = 5;
    int c = sum(a);
    return 0;
}
```

- Ошибка времени выполнения (runtime): фактически `sum` возьмёт что попало в качестве второго аргумента (то, что окажется в соответствующем регистре)

- Для решения этой проблемы существуют *заголовочные файлы* (*headers*, *хедеры*)

Заголовочные файлы

- Для решения этой проблемы существуют *заголовочные файлы (headers, хедеры)*
- В специальном файле с расширением `.h` помещается *объявление (declaration) функции без её реализации (определение, definition)*

```
// sum.h  
int sum(int x, int y);
```

Заголовочные файлы

```
// sum.c
#include "sum.h"

int sum(int x, int y) {
    return x + y;
}
```

```
// main.c
#include "sum.h"

int main() {
    int a = 3, b = 5;
    int c = sum(a);
    return 0;
}
```

- `#include` – директива препроцессора (одной из частей компилятора C)

Заголовочные файлы

- `#include` – директива препроцессора (одной из частей компилятора C)
- Она вставляет содержимое файла целиком в текущее место программы

```
// main.c после обработки препроцессором

int sum(int x, int y);

int main() {
    int a = 3, b = 5;
    int c = sum(a); // ошибка: неверное число аргументов
    return 0;
}
```

Заголовочные файлы

- `#include` – директива препроцессора (одной из частей компилятора C)
- Она вставляет содержимое файла целиком в текущее место программы

```
// main.c после обработки препроцессором

int sum(int x, int y);

int main() {
    int a = 3, b = 5;
    int c = sum(a); // ошибка: неверное число аргументов
    return 0;
}
```

- Имея *объявление функции*, компилятор может проверить соответствие типов аргументов и их количества